Лабораторная работа 10. Наивный байесовский классификатор

В этой лабораторной работе рассмотрим применение теории Байеса к задачам машинного обучения.

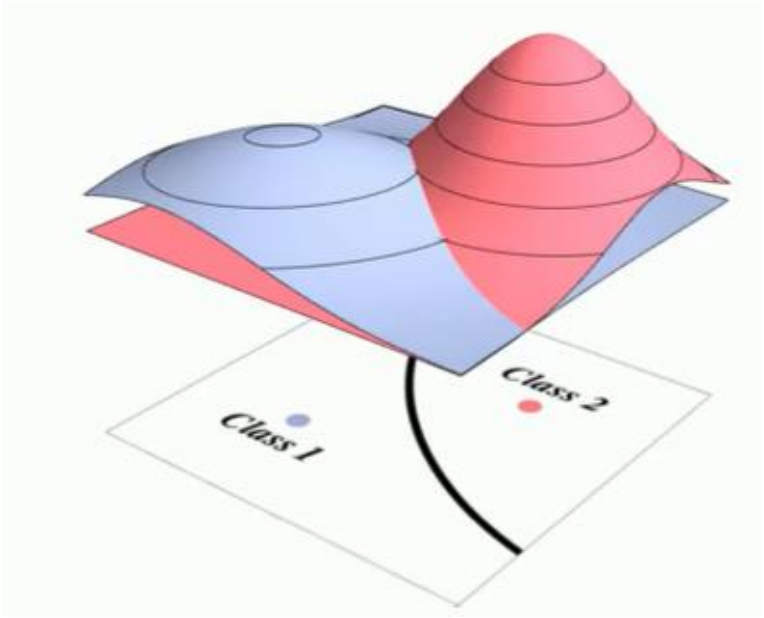
Рассмотрим задачу бинарной классификации. Выходами будут являться метками класса **-1** и **+1.**

$$y \in Y = \{-1; +1\}$$

При этом будет некий набор обучающей выборки $\boldsymbol{X^i}$, который подчиняется многомерному распределению ***p***

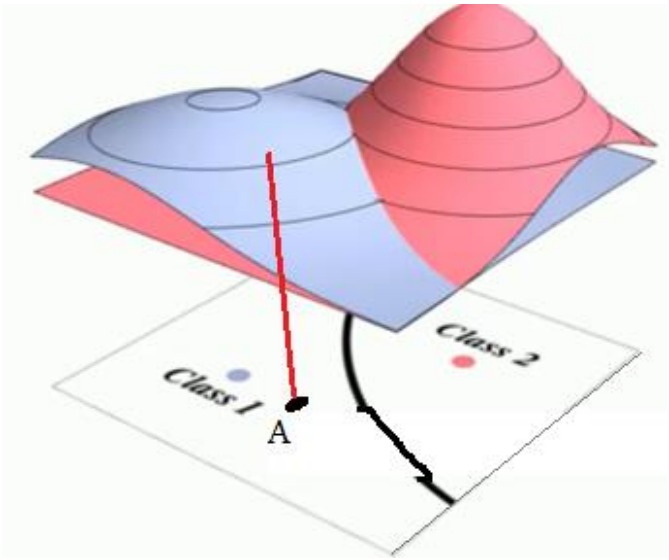
$$X^l = (x_i, y_i)_{i=1}^l \sim p(x, y)$$

Условно, плотность распределения вероятностей ***p*** можно представить графически.



Наша задача, каждую точку в этом признаковом пространстве, отнести либо к классу **-1**, либо к классу **+1**. Объект **x** следует отнести к тому классу, для которого вероятность наибольшая.

Пусть фиолетовая область пространства будет **Class1 (**метка **-1),** а розовая **Class2** (метка **+1)**. Если, мы попадаем в точку **A** признакового пространства, то логично отнести этот образ к **Class1** (метки классу **-1).**



Фактически нужно найти вероятности для классов **-1** и **+1 (y)** для конкретного образа **x (P(y|x)=?),** для конкретной точки в признаковом пространстве и выбрать тот класс, для которого эта условная вероятность окажется наибольшей**.**

Найти эту условную вероятность – это теорема Байес.

$$P(y \mid x) = \frac{P(y) \cdot p(x \mid y)}{p(x)}$$

P – обычная вероятность, **p** – плотность распределения вероятностей.

P(y) – это априорная вероятность появления класса **y.**

p(x|y) – условная плотность распределения вероятностей **x** от **y** - это распределение образов **x** для определенного класса **y,** т.е. для класса **-1,** когда **y = -1 –** распределение **x** – это фиолетовый график, а когда **y = +1** – это розовый график.

p(x) – это распределение образов без относительно их принадлежности к тому или иному классу.

Если мы знаем величины **P(y)** и **p(x|y),** тогда правило выбора класса в модели классификации можно записать следующим образом:

$$a(x) = \arg\max_{y \in Y} P(y \mid x)$$

Выбираем тот класс, вероятность которого окажется наибольшей или согласно формуле Байеса можно переписать условную вероятность следующим образом.

$$a(x) = \arg\max_{y \in Y} P(y \mid x) = \arg\max_{y \in Y} P(y) \cdot p(x \mid y)$$

Множитель ***p(x)*** отброшен, т.к. он не зависит от класса **y.**

Таким образом, мы поставили задачу классификации в машинном обучении согласно теореме Байеса. Причем число классов может быть сколько угодно. Ограничений нет.

На практики, в модель добавляют множитель **λ**, который **λ>0** или **λ=0.**

$$a(x) = \arg\max_{y \in Y} \lambda_y \cdot P(y) \cdot p(x \mid y)$$

Эти множители определяют величину штрафа при неверной классификации для каждого класса **y.**

Например, задача из военной тематики – радиолокационная станция для которой вероятность пропуск вражеского самолета, худшая ситуация по сравнению с «ложной тревогой». Штрафы должны быть разные. Когда цель пропускается штраф для **λ** больше, а когда срабатывает «ложная тревога», то штраф меньше.

Существует теорема, которая доказывает, что при правиле

$$a(x) = \arg\max_{y \in Y} \lambda_y \cdot P(y) \cdot p(x \mid y)$$

будут минимизироваться средние ошибки

$$\overline{R}(a) = \sum_{y \in Y} \lambda_y \int \left[a(x) \neq y \right] \cdot p(x, y)\, dx$$ - критерий оптимального байесовского классификатора

Минимизация данного критерия соответствует **оптимальному байесовскому классификатору.**

Из критерия видно, что если бы была известна плотность распределения вероятностей $\cdot p(x, y)$ образов **x** и целевых значений **y,** то задача классификации в машинном обучении была бы полностью решена. Однако, эта плотность распределения вероятностей неизвестна и ее приходится как-то оценивать или решить более простую задачу – это найти оценки этих величин:

$$\hat{P}(y),\ \hat{p}(x \mid y)$$

$\hat{P}(y)$ - это априорная вероятность появления того или иного класса.

$\hat{p}(x|y)$ - это плотность распределения вероятностей образов **x** в зависимости от класса **y**. Обычно такие оценки строятся по обучающей выборки.

После того, когда оценки найдены $\hat{P}(y), \hat{p}(x|y)$ и они подставлены байесовский классификатор $a(x) = \arg\max_{y \in Y} \lambda_y \cdot P(y) \cdot p(x|y)$, то он перестает быть оптимальным. Оптимальность здесь представляет скорее теоретический интерес. На практики, реальные классификаторы лишь могут приближаться к оптимальному.

Главная проблема при использовании байесовского классификатора – это оценить плотности распределения вероятностей .. $\hat{p}(x|y)$. Они многомерны.

В общем виде эта задача намного сложнее, чем построения классификатора с позиции выбора некоторых параметрических функций $a(x) = g(x, \theta)$ и последующего поиска векторов параметра θ.

Пусть $x = [\xi_1, \xi_2, \ldots, \xi_n]^T$, все признаки $\xi_1, \xi_2, \ldots, \xi_n$ независимы в вероятностном смысле, тогда многомерная плотность распределения вероятностей для них будет записана, как произведение соответствующих одномерных плотностей:

$$p(x|y) = p(\xi_1|y) \cdot p(\xi_2|y) \cdot \ldots \cdot p(\xi_n|y) = \prod_{i=1}^{n} p(\xi_i|y)$$

Такое упрощение будет соответствовать наивному байесовскому классификатору, который можно записать в виде:

$$a(x) = \arg\max_{y \in Y} \lambda_y \hat{P}(y) \prod_{i=1}^{n} \hat{p}(\xi_i|y)$$

Если возьмем логарифм, то наивный байесовский классификатор можно записать:

$$a(x) = \arg\max_{y \in Y} \left(\ln \lambda_y \hat{P}(y) + \sum_{i=1}^{n} \ln \hat{p}(\xi_i \mid y) \right)$$

В этом выражении, мы оперируем $\hat{p}(\xi_i \mid y)$ одномерными плотностями распределения вероятности, что значительно упрощает задачу. Восстановить одномерную плотность гораздо проще, чем многомерную.

Часто, алгоритм наивного байесовского классификатора реализуют с использованием Гауссовских плотностей распределения вероятностей $\hat{p}(\xi_i \mid y)$, т.е. предполагают, что все признаки $\xi_1, \xi_2, \ldots, \xi_n$ распределены по Гауссовом закону. Это распространённый случай, но не всегда признаки подчиняются нормальному закону распределения. Если в решаемой задачи это не так (плотность распределения вероятности признаков иная), то использование нормального распределения может привести к значительным ошибкам. В этом случае необходимо выбрать другой вид плотности распределения.

Рассмотрим пример использования наивного байесовского классификатора. Пусть существует набор данных (обучающая выборка).

№	Ширина	Длина	Жук	
1	10	50	гусеница	-1
2	20	30	божья коровка	+1
3	25	30	божья коровка	+1
4	20	60	гусеница	-1
5	15	70	гусеница	-1
6	40	40	божья коровка	+1
7	30	45	божья коровка	+1
8	20	45	гусеница	-1
9	40	30	божья коровка	+1
10	7	35	гусеница	-1

Будем предполагать, что ширина и длина – это два независимых признака, который подчиняются нормальному закону распределению. Нормальный закон распределения для одномерной случайной величины:

$$p(l|y)=\frac{1}{\sqrt{2\pi}\sigma_l}\exp\left\{-\frac{(l-m_l)^2}{2\sigma_l^2}\right\}$$

$$p(w|y)=\frac{1}{\sqrt{2\pi}\sigma_w}\exp\left\{-\frac{(w-m_w)^2}{2\sigma_w^2}\right\}$$

Случайные величины – длина (***l***) и ширина (***w***). Причем будем эти параметры определять в зависимости от класса жука (**y**) – отдельно для гусениц и отдельно для божьих коровок.

Параметры σ_l^2 , σ_w^2 - это дисперсия случайных величин ***l***, ***w***.

m_l, m_w - математические ожидания для величин ***l***, ***w***.

Чтобы восстановить плотности распределения вероятностей $p(l|y)$, $p(w|y)$ нужно оценит математическое ожидание и дисперсию.

Для математических ожиданий можно записать

$$\hat{m}_l^{(-1)}=\frac{1}{5}\cdot(50+60+70+45+35)=52$$

$$\hat{m}_l^{(+1)}=\frac{1}{5}\cdot(30+30+40+45+30)=35$$

$$\hat{m}_w^{(-1)}=\frac{1}{5}\cdot(10+20+15+20+7)=14,4$$

$$\hat{m}_w^{(+1)}=\frac{1}{5}\cdot(20+25+40+30+40)=31$$

Вычисления проводим для класса **-1** и **+1**.

Далее по формуле вычисляем дисперсию:

$$\hat{\sigma}_x^2 = \frac{1}{N-1}\sum_{i=1}^{N}\left(x_i - \hat{m}_x\right)^2$$

Вычисления проводим для класса **-1** и **+1**.

$$\hat{\sigma}^2_{l(-1)} = \frac{1}{4}\cdot\left((50-52)^2+(60-52)^2+(70-52)^2+(45-52)^2+(35-52)^2\right)=182,5$$

$$\hat{\sigma}^2_{l(+1)} = \frac{1}{4}\cdot\left((30-35)^2+(30-35)^2+(40-35)^2+(45-35)^2+(30-35)^2\right)=50$$

$$\hat{\sigma}^2_{w(-1)} = \frac{1}{4}\cdot\left((10-14,4)^2+(20-14,4)^2+(15-14,4)^2+(20-14,4)^2+(7-14,4)^2\right)=34,4$$

$$\hat{\sigma}^2_{w(+1)} = \frac{1}{4}\cdot\left((20-31)^2+(25-31)^2+(40-31)^2+(30-31)^2+(40-31)^2\right)=80$$

Далее, на основе вычисленных значений можно восстановить плотность распределения вероятности образов **x** в зависимости от номера класса **y**.

$$p(x\,|\,y) = p(l,w\,|\,y) = p(l\,|\,y)\cdot p(w|\,y)$$

В данном случае вектор **x** зависит от двух параметров – это длина и ширина жуков. Учитывая, что эти параметры независимы, то условная плотность распределения может быть расписана как произведение соответствующих одномерных плотности. Расписывая эту плотность для Гауссовских распределений, мы получаем для соответствующих классов **-1** и **+1**:

$$\hat{p}(l,w\,|\,y=-1) = \frac{1}{2\pi\hat{\sigma}_{l(-1)}\hat{\sigma}_{w(-1)}}\cdot\exp\left\{-\frac{\left(l-\hat{m}_{l(-1)}\right)^2}{2\hat{\sigma}^2_{l(-1)}}-\frac{\left(w-\hat{m}_{w(-1)}\right)^2}{2\hat{\sigma}^2_{w(-1)}}\right\}$$

$$\hat{p}(l,w\,|\,y=+1) = \frac{1}{2\pi\hat{\sigma}_{l(+1)}\hat{\sigma}_{w(+1)}}\cdot\exp\left\{-\frac{\left(l-\hat{m}_{l(+1)}\right)^2}{2\hat{\sigma}^2_{l(+1)}}-\frac{\left(w-\hat{m}_{w(+1)}\right)^2}{2\hat{\sigma}^2_{w(+1)}}\right\}$$

Осталось определить априорную вероятность появления объектов классов **-1** и **+1**. Для этого подсчитываем количество образов, которые относятся к классу **-1** и

+1, их у нас одинаковое количество (5 штук) и делим на общее количество образов в выборки 10.

$$\hat{P}(y=-1)=\hat{P}(y=+1)=\frac{5}{10}=\frac{1}{2}$$

В итоги, оценка априорных вероятностей для классов **-1** и **+1** одинаковы и составляют $\frac{1}{2}$ или 0,5.

Все данные подсчитаны, теперь можно применить наивный байесовский классификатор для определения класса к которому относится вектор **x** по следующей формуле.

$$a(x)=\arg\max_{y\in Y}\left(\ln\lambda_y\hat{P}(y)+\sum_{i=1}^{n}\ln\hat{p}(\xi_i\mid y)\right)$$

Подставляем (в ξ_i) конкретную ширину и длину жуков и выбираем то выражение, для которого получаем максимальное значение.

Величины штрафа берем одинаковые $\lambda_{y=-1}=\lambda_{y=+1}=1$.

Далее расписывая формулу $\left(\ln\lambda_y\hat{P}(y)+\sum_{i=1}^{n}\ln\hat{p}(\xi_i\mid y)\right)$ мы получаем.

$$\begin{cases} a(l,w\mid y=-1)=\ln\frac{1}{2}+\ln\left(\frac{1}{2\pi\hat{\sigma}_{l(-1)}\hat{\sigma}_{w(-1)}}\right)+\left(-\frac{\left(l-\hat{m}_{l(-1)}\right)^2}{2\hat{\sigma}^2_{l(-1)}}-\frac{\left(w-\hat{m}_{w(-1)}\right)^2}{2\hat{\sigma}^2_{w(-1)}}\right) \\ a(l,w\mid y=+1)=\ln\frac{1}{2}+\ln\left(\frac{1}{2\pi\hat{\sigma}_{l(+1)}\hat{\sigma}_{w(+1)}}\right)+\left(-\frac{\left(l-\hat{m}_{l(+1)}\right)^2}{2\hat{\sigma}^2_{l(+1)}}-\frac{\left(w-\hat{m}_{w(+1)}\right)^2}{2\hat{\sigma}^2_{w(+1)}}\right) \end{cases}$$

Вместо **x** подставляем ***l, w*** – это длина и ширина жуков для классов **-1** и **+1** ($a(l,w\mid y=-1)$ и $a(l,w\mid y=+1)$).

Поскольку $\lambda_{y=-1}=\lambda_{y=+1}=1$, а $\hat{P}(y=-1)=\hat{P}(y=+1)=\frac{1}{2}$, то получаем $ln1/2$.

Выражение $\sum_{i=1}^{n}\ln\hat{p}(\xi_i \mid y)$ переписываем из условия

$$\hat{p}(l,w \mid y=-1)=\frac{1}{2\pi\hat{\sigma}_{l(-1)}\hat{\sigma}_{w(-1)}}\cdot\exp\left\{-\frac{\left(l-\hat{m}_{l(-1)}\right)^2}{2\hat{\sigma}^2_{l(-1)}}-\frac{\left(w-\hat{m}_{w(-1)}\right)^2}{2\hat{\sigma}^2_{w(-1)}}\right\}$$

$$\hat{p}(l,w \mid y=+1)=\frac{1}{2\pi\hat{\sigma}_{l(+1)}\hat{\sigma}_{w(+1)}}\cdot\exp\left\{-\frac{\left(l-\hat{m}_{l(+1)}\right)^2}{2\hat{\sigma}^2_{l(+1)}}-\frac{\left(w-\hat{m}_{w(+1)}\right)^2}{2\hat{\sigma}^2_{w(+1)}}\right\}$$

и получаем:

$$\ln\left(\frac{1}{2\pi\hat{\sigma}_{l(-1)}\hat{\sigma}_{w(-1)}}\right)+\left(-\frac{\left(l-\hat{m}_{l(-1)}\right)^2}{2\hat{\sigma}^2_{l(-1)}}-\frac{\left(w-\hat{m}_{w(-1)}\right)^2}{2\hat{\sigma}^2_{w(-1)}}\right)$$

$$\ln\left(\frac{1}{2\pi\hat{\sigma}_{l(+1)}\hat{\sigma}_{w(+1)}}\right)+\left(-\frac{\left(l-\hat{m}_{l(+1)}\right)^2}{2\hat{\sigma}^2_{l(+1)}}-\frac{\left(w-\hat{m}_{w(+1)}\right)^2}{2\hat{\sigma}^2_{w(+1)}}\right)$$

Таким образом, формула для классов -1 и +1 будет:

$$\begin{cases} a(l,w|\,y=-1) = \ln\frac{1}{2} + \ln\left(\frac{1}{2\pi\hat{\sigma}_{l(-1)}\hat{\sigma}_{w(-1)}}\right) + \left(-\frac{\left(l-\hat{m}_{l(-1)}\right)^2}{2\hat{\sigma}^2_{l(-1)}} - \frac{\left(w-\hat{m}_{w(-1)}\right)^2}{2\hat{\sigma}^2_{w(-1)}}\right) \\ a(l,w|\,y=+1) = \ln\frac{1}{2} + \ln\left(\frac{1}{2\pi\hat{\sigma}_{l(+1)}\hat{\sigma}_{w(+1)}}\right) + \left(-\frac{\left(l-\hat{m}_{l(+1)}\right)^2}{2\hat{\sigma}^2_{l(+1)}} - \frac{\left(w-\hat{m}_{w(+1)}\right)^2}{2\hat{\sigma}^2_{w(+1)}}\right) \end{cases}$$

Далее, упрощаем выражения. Поскольку берется **argmax**, то одинаковые слагаемые на положении точки максимум никак не сказываются. Выражение

$$\begin{cases} a(l,w|\,y=-1) = \ln\frac{1}{2} + \ln\left(\frac{1}{2\pi\hat{\sigma}_{l(-1)}\hat{\sigma}_{w(-1)}}\right) + \left(-\frac{\left(l-\hat{m}_{l(-1)}\right)^2}{2\hat{\sigma}^2_{l(-1)}} - \frac{\left(w-\hat{m}_{w(-1)}\right)^2}{2\hat{\sigma}^2_{w(-1)}}\right) \\ a(l,w|\,y=+1) = \ln\frac{1}{2} + \ln\left(\frac{1}{2\pi\hat{\sigma}_{l(+1)}\hat{\sigma}_{w(+1)}}\right) + \left(-\frac{\left(l-\hat{m}_{l(+1)}\right)^2}{2\hat{\sigma}^2_{l(+1)}} - \frac{\left(w-\hat{m}_{w(+1)}\right)^2}{2\hat{\sigma}^2_{w(+1)}}\right) \end{cases}$$

Выражение $\ln\left(\frac{1}{2\pi\hat{\sigma}_{l(-1)}\hat{\sigma}_{w(-1)}}\right)$ можно переписать, как $-\ln\left(2\pi\hat{\sigma}_{l(-1)}\hat{\sigma}_{w(-1)}\right)$ при этом 2π можно убрать, как одинаковые слагаемые. Тогда, окончательно получим выражение, которое позволит классифицировать жуков:

$$\begin{cases} a(l,w|\,y=-1) = -\ln\left(\hat{\sigma}_{l(-1)}\hat{\sigma}_{w(-1)}\right) - \frac{\left(l-\hat{m}_{l(-1)}\right)^2}{2\hat{\sigma}^2_{l(-1)}} - \frac{\left(w-\hat{m}_{w(-1)}\right)^2}{2\hat{\sigma}^2_{w(-1)}} \\ a(l,w|\,y=+1) = -\ln\left(\hat{\sigma}_{l(+1)}\hat{\sigma}_{w(+1)}\right) - \frac{\left(l-\hat{m}_{l(+1)}\right)^2}{2\hat{\sigma}^2_{l(+1)}} - \frac{\left(w-\hat{m}_{w(+1)}\right)^2}{2\hat{\sigma}^2_{w(+1)}} \end{cases}$$

Реализуем алгоритм на языку Python. Вначале загрузим необходимую библиотеку.

Запишем обучающую выборку – это длина и ширина жука:

x_train = np.array([[10, 50], [20, 30], [25, 30], [20, 60], [15, 70], [40, 40], [30, 45], [20, 45], [40, 30], [7, 35]])

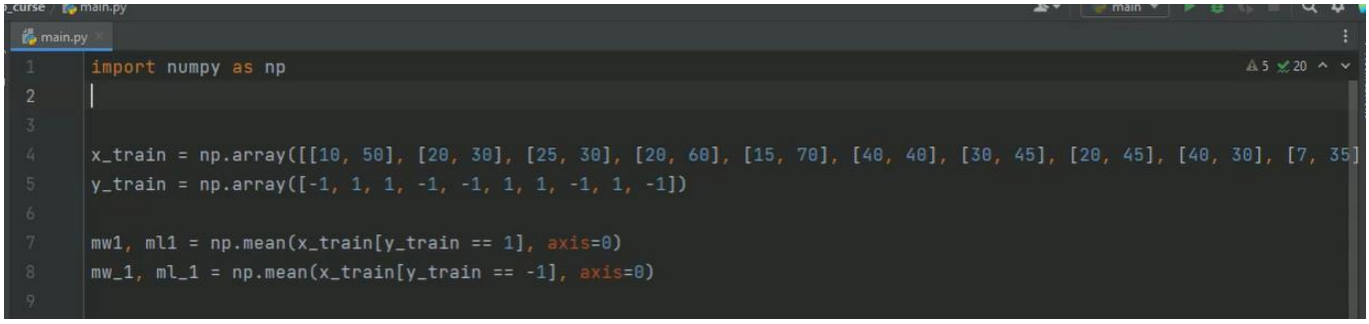
Далее метки классов – это **-1** и **1**

y_train = np.array([-1, 1, 1, -1, -1, 1, 1, -1, 1, -1])

Далее вычисляем математическое ожидание для класса **1** и класса **-1**

mw1, ml1 = np.mean(x_train[y_train == 1], axis=0)

mw_1, ml_1 = np.mean(x_train[y_train == -1], axis=0)



Далее вычисляем дисперсию для класса **1** и класса **-1**

sw1, sl1 = np.var(x_train[y_train == 1], axis=0, ddof=1)

sw_1, sl_1 = np.var(x_train[y_train == -1], axis=0, ddof=1)

Далее выводим на экран все вычисленные значения математического ожидания и дисперсии

print('МО: ', mw1, ml1, mw_1, ml_1)

print('Дисперсии:', sw1, sl1, sw_1, sl_1)

```
9
10  # формула для вычисления дисперсии здесь немного другая 1/N*sum(...)
11  sw1, sl1 = np.var(x_train[y_train == 1], axis=0, ddof=1)
12  sw_1, sl_1 = np.var(x_train[y_train == -1], axis=0, ddof=1)
13
14  print('M0: ', mw1, ml1, mw_1, ml_1)
15  print('Дисперсии:', sw1, sl1, sw_1, sl_1)
16
```

Далее формируем вектор **x** с шириной и длиной жука

x = [10, 40] # ширина, длина жука

Записываем через **lambda** функции модели для класса **-1** и класса **1.**

a_1 = lambda x: -np.log(sw_1 * sl_1) - (x[0] - mw_1) ** 2 / (2 * sw_1) - (x[1] - ml_1) ** 2 / (2 * sl_1)

a1 = lambda x: -np.log(sw1 * sl1) - (x[0] - mw1) ** 2 / (2 * sw1) - (x[1] - ml1) ** 2 / (2 * sl1)

Передается один аргумент **x** и вычисляется по формуле соответственно для класса **-1** и класса **+1** выходные значения для модели ***a***.

$$\begin{cases} a(l,w|\,y=-1) = -\ln\left(\hat{\sigma}_{l(-1)}\hat{\sigma}_{w(-1)}\right) - \dfrac{\left(l-\hat{m}_{l(-1)}\right)^2}{2\hat{\sigma}^2_{l(-1)}} - \dfrac{\left(w-\hat{m}_{w(-1)}\right)^2}{2\hat{\sigma}^2_{w(-1)}} \\ a(l,w|\,y=+1) = -\ln\left(\hat{\sigma}_{l(+1)}\hat{\sigma}_{w(+1)}\right) - \dfrac{\left(l-\hat{m}_{l(+1)}\right)^2}{2\hat{\sigma}^2_{l(+1)}} - \dfrac{\left(w-\hat{m}_{w(+1)}\right)^2}{2\hat{\sigma}^2_{w(+1)}} \end{cases}$$

Далее, относим вектор **x** к тому классу, для которого модель выдала наибольшее значения.

y = np.argmax([a_1(x), a1(x)]) * 2 – 1

np.argmax([a_1(x), a1(x)]) – значение будет либо **0** либо **1**, но чтобы получить метки класса **+1** или **-1** умножим на **2** и вычтем **-1**.

Далее, выводим на экран ту метку класса, которую получили для вектора **x** (**x = [10, 40] # ширина, длина жука**).

print('Номер класса (-1 - гусеница, 1 - божья коровка): ', y)

```
x = [10, 40]  # ширина, длина жука

a_1 = lambda x: -np.log(sw_1 * sl_1) - (x[0] - mw_1) ** 2 / (2 * sw_1) - (x[1] - ml_1) ** 2 / (2 * sl_1)
a1 = lambda x: -np.log(sw1 * sl1) - (x[0] - mw1) ** 2 / (2 * sw1) - (x[1] - ml1) ** 2 / (2 * sl1)
y = np.argmax([a_1(x), a1(x)]) * 2 - 1

print('Номер класса (-1 - гусеница, 1 - божья коровка): ', y)
```

Пропускаем по обучающей выборки и определяем долю ошибок, которых совершил наивный байесовский классификатор.

тестирование по обучающей выборки

pr = []

for x in x_train:

pr.append(np.argmax([a_1(x), a1(x)]) * 2 - 1)

pr = np.array(pr)

Q = np.mean(pr != y_train) # доля ошибок

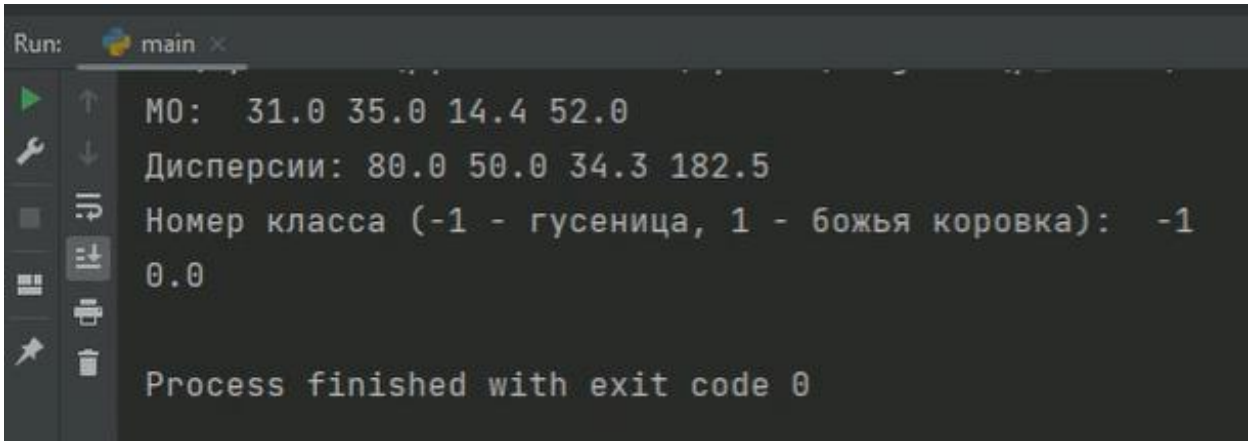
print(Q)

```
print('Номер класса (-1 - гусеница, 1 - божья коровка): ', y)

# тестирование по обучающей выборки
pr = []
for x in x_train:
    pr.append(np.argmax([a_1(x), a1(x)]) * 2 - 1)

pr = np.array(pr)
Q = np.mean(pr != y_train) # доля ошибок
print(Q)
```

Запускаем программу. Результаты математического ожидания и дисперсии совпадают с результатами, полученными вручную.



$$\hat{m}_l^{(-1)} = \frac{1}{5}\cdot(50+60+70+45+35) = 52$$

$$\hat{m}_l^{(+1)} = \frac{1}{5}\cdot(30+30+40+45+30) = 35$$

$$\hat{m}_w^{(-1)} = \frac{1}{5}\cdot(10+20+15+20+7) = 14,4$$

$$\hat{m}_w^{(+1)} = \frac{1}{5}\cdot(20+25+40+30+40) = 31$$

$$\hat{\sigma}^2_{l(-1)} = \frac{1}{4}\cdot\left((50-52)^2+(60-52)^2+(70-52)^2+(45-52)^2+(35-52)^2\right) = 182,5$$

$$\hat{\sigma}^2_{l(+1)} = \frac{1}{4}\cdot\left((30-35)^2+(30-35)^2+(40-35)^2+(45-35)^2+(30-35)^2\right) = 50$$

$$\hat{\sigma}^2_{w(-1)} = \frac{1}{4}\cdot\left((10-14,4)^2+(20-14,4)^2+(15-14,4)^2+(20-14,4)^2+(7-14,4)^2\right) = 34,4$$

$$\hat{\sigma}^2_{w(+1)} = \frac{1}{4}\cdot\left((20-31)^2+(25-31)^2+(40-31)^2+(30-31)^2+(40-31)^2\right) = 80$$

Для жука с параметрами **x = [10, 40]** – результат **-1 – гусеница**. При этом доля ошибок – неверных классификаций равна **0**.

Наивный байесовский классификатор точно классифицировал всю обучающую выборку **([[10, 50], [20, 30], [25, 30], [20, 60], [15, 70], [40, 40], [30, 45], [20, 45], [40, 30], [7, 35]]**).

```python
# Пример реализации наивного гауссовского байесовского классификатора

import numpy as np


x_train = np.array([[10, 50], [20, 30], [25, 30], [20, 60], [15, 70], [40, 40], [30, 45], [20, 45], [40, 30], [7, 35]])
y_train = np.array([-1, 1, 1, -1, -1, 1, 1, -1, 1, -1])

mw1, ml1 = np.mean(x_train[y_train == 1], axis=0)
mw_1, ml_1 = np.mean(x_train[y_train == -1], axis=0)

# формула для вычисления дисперсии здесь немного другая 1/N*sum(...)
sw1, sl1 = np.var(x_train[y_train == 1], axis=0, ddof=1)
sw_1, sl_1 = np.var(x_train[y_train == -1], axis=0, ddof=1)

print('МО: ', mw1, ml1, mw_1, ml_1)
print('Дисперсии:', sw1, sl1, sw_1, sl_1)

x = [10, 40]  # ширина, длина жука

a_1 = lambda x: -np.log(sw_1 * sl_1) - (x[0] - mw_1) ** 2 / (2 * sw_1) - (x[1] - ml_1) ** 2 / (2 * sl_1)
a1 = lambda x: -np.log(sw1 * sl1) - (x[0] - mw1) ** 2 / (2 * sw1) - (x[1] - ml1) ** 2 / (2 * sl1)
y = np.argmax([a_1(x), a1(x)]) * 2 - 1

print('Номер класса (-1 - гусеница, 1 - божья коровка): ', y)

```

```
24
25      print('Номер класса (-1 - гусеница, 1 - божья коровка): ', y)
26
27      # тестирование по обучающей выборки
28      pr = []
29      for x in x_train:
30          pr.append(np.argmax([a_1(x), a1(x)]) * 2 - 1)
31
32      pr = np.array(pr)
33      Q = np.mean(pr != y_train) # доля ошибок
34      print(Q)
```

Задание для самостоятельного выполнения:

1. Изучить материал.
2. Составить самостоятельно обучающую выборку (таблицу) для двух (можно и более 3...5) сущностей.
3. Выполнить классификацию с использованием наивного байесовского классификатора.